МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИИТ)
Кафедра цифровой трансформации (ЦТ)

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ

по дисциплине «Разработка баз данных»

Практическая работа № 7

Студент группы *ИВБО-20-24 Дьяков Андрей Геннадьевич* _____
(подпись)

Ассистент *Шерри Георгий Даниэлевич*

_____
(подпись)

Отчет представлен «___»_____2026 г.

Москва 2026 г.

ОПТИМИЗАЦИЯ ЗАПРОСОВ, ИНДЕКСЫ И ОБСЛУЖИВАНИЕ БАЗЫ ДАННЫХ В POSTGRESQL

Цель работы: Целью данной практической работы является формирование у студентов глубоких теоретических знаний и практических навыков анализа производительности SQL-запросов, а также освоение полного спектра инструментов оптимизации в СУБД PostgreSQL.

ПОДГОТОВКА БАЗЫ ДАННЫХ

Для выполнения работы используется база данных туристического агентства. Исследуются три таблицы: **service** (услуги: название, тип услуги, страна, поставщик, авиакомпания, туроператор), **customer_order** (заказы клиентов: дата создания, клиент, сотрудник, статус, дата оплаты, сумма) и **user_logs** (журнал действий пользователей: дата, пользователь, JSONB-документ event_data и IP-адрес). Для демонстрации специфических индексов в таблицу user_logs добавлен столбец ip_address типа inet.

Чтобы оптимизатор PostgreSQL не выбирал Seq Scan только из-за малого размера таблиц, в каждую из них скриптом (листинг 1) добавлено по 20 000 строк. Внешние ключи берутся из существующих записей родительских таблиц (country, supplier, airline, tour_operator), поэтому нарушение ссылочной целостности исключено. Значения генерируются так, чтобы на них можно было показать работу разных индексов:

- client_id в заказах принимает 10 значений ((i % 10) + 1), поэтому условие client_id = 3 выбирает около 10 % строк customer_order;
- дата оплаты paid_at равна NULL у каждого третьего заказа (около 33 %);
- created_at и total_amount заданы функцией random(), поэтому физический порядок строк customer_order не связан со значениями этих столбцов;

— в массиве tags JSONB-документа user_logs значение «promo» встречается у каждой третьей записи, а ip_address формируется в подсети 192.168.0.0/16.

Листинг 1. Скрипт генерации тестовых данных и проверки количества строк

```
ALTER TABLE user_logs
ADD COLUMN IF NOT EXISTS ip_address inet;

WITH
country_ids AS (
    SELECT array_agg(id ORDER BY id) AS ids
    FROM country
),
supplier_ids AS (
    SELECT array_agg(id ORDER BY id) AS ids
    FROM supplier
),
airline_ids AS (
    SELECT array_agg(id ORDER BY id) AS ids
    FROM airline
),
tour_operator_ids AS (
    SELECT array_agg(id ORDER BY id) AS ids
    FROM tour_operator
)
INSERT INTO service (
    name,
    service_type,
    country_id,
    departure_city,
    supplier_id,
    description,
    airline_id,
    tour_operator_id
)
SELECT
    'Тур №' || i,
    CASE
        WHEN i % 4 = 0 THEN 'flight'
        WHEN i % 4 = 1 THEN 'tour'
        WHEN i % 4 = 2 THEN 'hotel'
        ELSE 'insurance'
    END,
    country_ids.ids[(i % cardinality(country_ids.ids)) + 1],
    CASE
        WHEN i % 4 = 0 THEN 'Москва (MOW)'
        ELSE NULL
    END,
    supplier_ids.ids[(i % cardinality(supplier_ids.ids)) + 1],
    'Описание услуги ' || i,
    airline_ids.ids[(i % cardinality(airline_ids.ids)) + 1],
    tour_operator_ids.ids[(i % cardinality(tour_operator_ids.ids)) + 1]
FROM generate_series(1, 20000) AS i
CROSS JOIN country_ids
CROSS JOIN supplier_ids
CROSS JOIN airline_ids
CROSS JOIN tour_operator_ids;

INSERT INTO customer_order (
    created_at,
```

```
    client_id,
    employee_id,
    status_id,
    paid_at,
    total_amount
)
SELECT
    created_at,
    client_id,
    employee_id,
    status_id,
    CASE
        WHEN is_paid THEN created_at + INTERVAL '1 hour'
        ELSE NULL
    END,
    total_amount
FROM (
    SELECT
        NOW() - (random() * INTERVAL '1000 days') AS created_at,
        (i % 10) + 1 AS client_id,
        ((i * 7) % 10) + 1 AS employee_id,
        ((i * 3) % 10) + 1 AS status_id,
        (i % 3 <> 0) AS is_paid,
        round((random() * 490000 + 10000)::numeric, 2) AS total_amount
    FROM generate_series(1, 20000) AS i
) x;

INSERT INTO user_logs (
    created_at,
    user_id,
    event_data,
    ip_address
)
SELECT
    NOW() - INTERVAL '1 second' * (20000 - i),
    (i % 1000) + 1,
    jsonb_build_object(
        'event',
            CASE
                WHEN i % 5 = 0 THEN 'booking'
                WHEN i % 5 = 1 THEN 'login'
                WHEN i % 5 = 2 THEN 'view_tour'
                WHEN i % 5 = 3 THEN 'search'
                ELSE 'logout'
            END,
        'user_id', (i % 1000) + 1,
        'tour_id', (i % 20000) + 1,
        'amount', round((random() * 200000)::numeric, 2),
        'success', (random() > 0.2),
        'tags',
            CASE
                WHEN i % 3 = 0 THEN jsonb_build_array('promo', 'sale')
                WHEN i % 3 = 1 THEN jsonb_build_array('news')
                ELSE jsonb_build_array('updates', 'alert')
            END
    ),
    (
        '192.168.' ||
        ((i % 254) + 1) ||
        '.' ||
        (((i / 254) % 254) + 1)
    )::inet
FROM generate_series(1, 20000) AS i;
```

```
SELECT
    (SELECT COUNT(*) FROM service)        AS service_count,
    (SELECT COUNT(*) FROM customer_order) AS order_count,
    (SELECT COUNT(*) FROM user_logs)      AS user_logs_count;
```

После выполнения скрипта проверено число строк в каждой таблице (рисунок 1).

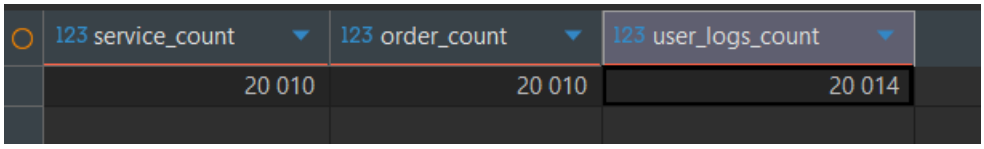


Рисунок 1. Результат выполнения запросов SELECT COUNT(*), подтверждающий наличие более 20 000 строк в каждой таблице

Проводим анализ:

В таблицах service, customer_order и user_logs теперь 20 010, 20 010 и 20 014 строк (исходно было 10, 10 и 14; добавлено по 20 000). Это превышает требуемый минимум, поэтому оптимизатор уже рассматривает индексные методы доступа.

Методика замеров.

Все запросы выполняются командой EXPLAIN (ANALYZE, BUFFERS). Сравниваются: узел плана, Rows Removed by Filter, число блоков (Buffers: shared hit + read; один блок — 8 КБ), Planning Time и Execution Time. Основной показатель — число прочитанных блоков: время зависит от кэша операционной системы и загрузки сервера, а число блоков для конкретного плана остаётся неизменным.

Задание №1: анализ и оптимизация с использованием B-Tree

1.1 Простой B-Tree (стандартный поиск)

Задача. Найти в таблице service услугу по точному названию «Тур №15000».

Шаг 1. Анализ «ДО» (без индекса)

Выполняем профилирование запроса с включённым сбором статистики по буферам (листинг 2).

Листинг 2. Анализ запроса с поиском по точному названию

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM service WHERE name = 'Тур №15000';
```

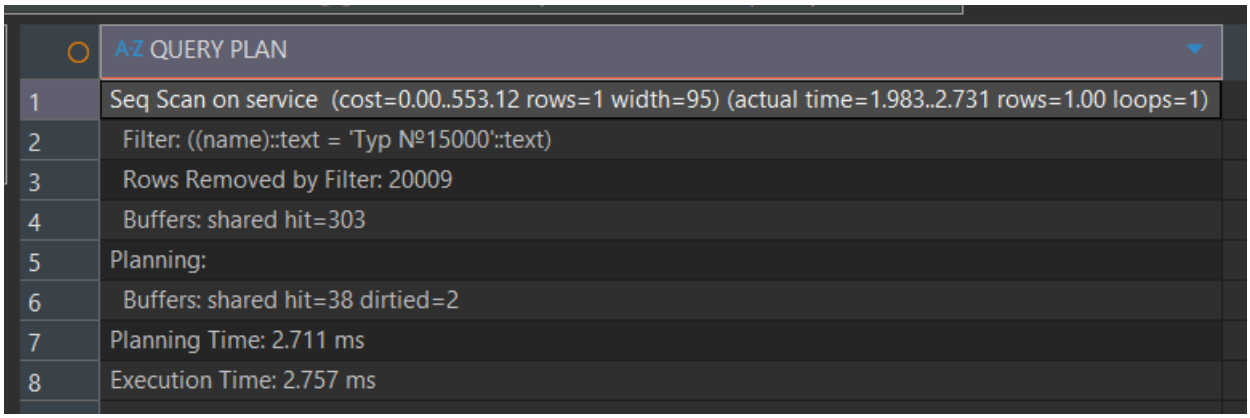


Рисунок 2. План выполнения Seq Scan до создания B-Tree индекса

Проводим анализ плана «ДО»:

— Узел плана: **Seq Scan on service**. Индекса по столбцу name нет, поэтому PostgreSQL последовательно читает всю таблицу.

— Фильтрация: условие name = 'Тур №15000' проверено для каждой строки; отброшено **20 009 строк** (Rows Removed by Filter: 20009), чтобы вернуть единственную.

— Буферы: **shared hit = 303**, то есть прочитано 303 блока по 8 КБ (около 2,4 МБ) ради одной записи.

— Время: Execution Time = 2,757 мс, Planning Time = 2,711 мс; оценка стоимости плана — cost 0.00..553.12.

Шаг 2. Создание индекса

Создаём стандартный B-Tree индекс по столбцу name (ключевое слово USING btree подразумевается по умолчанию).

Листинг 3. Создание B-Tree индекса

```
CREATE INDEX idx_service_name ON service (name);
```

Шаг 3. Анализ «ПОСЛЕ» (с индексом)

Выполняем тот же самый запрос ещё раз (листинг 4).

Листинг 4. Повторный анализ запроса

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT * FROM service WHERE name = 'Typ №15000';
```

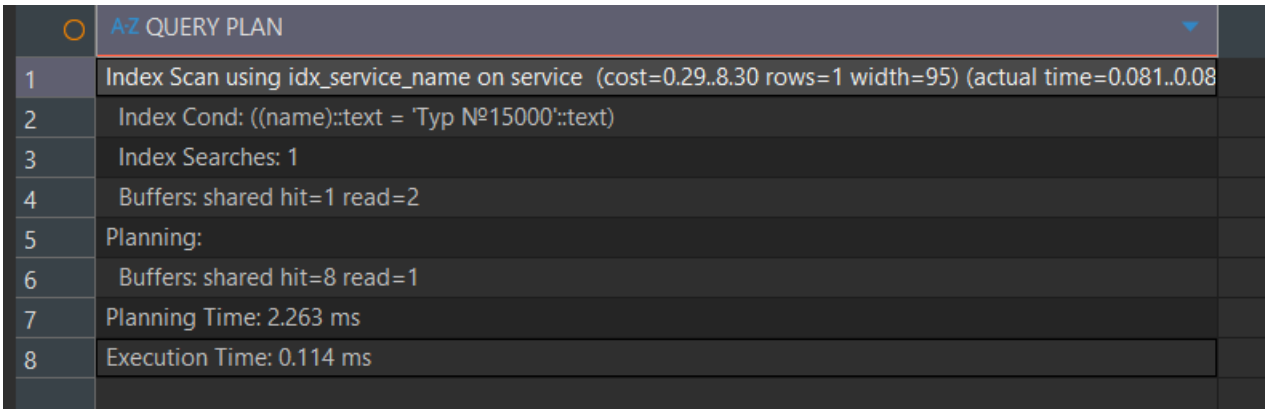


Рисунок 3. План выполнения Index Scan после создания B-Tree индекса

Проводим анализ плана «ПОСЛЕ»:

— Узел плана: **Index Scan using idx_service_name** (cost 0.29..8.30, Index Searches: 1). PostgreSQL спустился по дереву индекса, получил TID и прочитал только одну нужную строку.

— Строка Rows Removed by Filter **исчезла**: лишние строки больше не читаются.

— Буферы: shared hit = 1, read = 2, всего **3 блока** (страницы индекса и страница таблицы) вместо 303 — нагрузка на ввод-вывод снизилась примерно в 100 раз. Два блока отсутствовали в shared buffers и были прочитаны с диска (или из кэша ОС).

— Время: Execution Time = 0,114 мс против 2,757 мс (быстрее в ≈24 раза). Planning Time (2,263 мс) стал основной частью общего времени запроса, поэтому итоговый выигрыш (5,468 → 2,377 мс) составил ≈2,3 раза. С ростом таблицы Execution Time без индекса растёт пропорционально числу строк, а с индексом почти не меняется.

Шаг 4. Сравнительная таблица

Таблица 1. Сравнение показателей запроса до и после создания B-Tree индекса

Метрика	До оптимизации	После оптимизации	Вывод
Узел плана	Seq Scan	Index Scan	Полный перебор заменён адресным доступом по индексу

Метрика	До оптимизации	После оптимизации	Вывод
Rows Removed by Filter	20 009	0	Строки больше не читаются и не отбрасываются
Buffers: shared hit	303	1	Почти все данные для ответа не требуются
Buffers: read	0	2	Обращения к диску минимальны
Всего блоков (hit + read)	303 (≈2,4 МБ)	3 (24 КБ)	Нагрузка на ввод-вывод снизилась в ≈100 раз
Planning Time	2,711 мс	2,263 мс	Изменение на уровне погрешности измерения
Execution Time	2,757 мс	0,114 мс	Выполнение ускорилось в ≈24 раза
Итого (Planning + Execution)	5,468 мс	2,377 мс	Общий выигрыш ≈2,3 раза

Вывод. B-Tree индекс по столбцу точного поиска сократил число читаемых блоков с 303 до 3, а время выполнения запроса — в ≈24 раза. Чем больше таблица, тем заметнее разница.

1.2 Составной B-Tree индекс: правило левого префикса

Задача. Найти в таблице customer_order заказы клиента №3 с суммой от 50 000 до 150 000. Условие client_id = 3 — точное совпадение (равенство), условие total_amount BETWEEN 50000 AND 150000 — диапазон.

Шаг 1. Анализ «ДО» (без индекса)

Листинг 5. Анализ запроса с составным условием

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM customer_order
WHERE client_id = 3
  AND total_amount BETWEEN 50000 AND 150000;
```

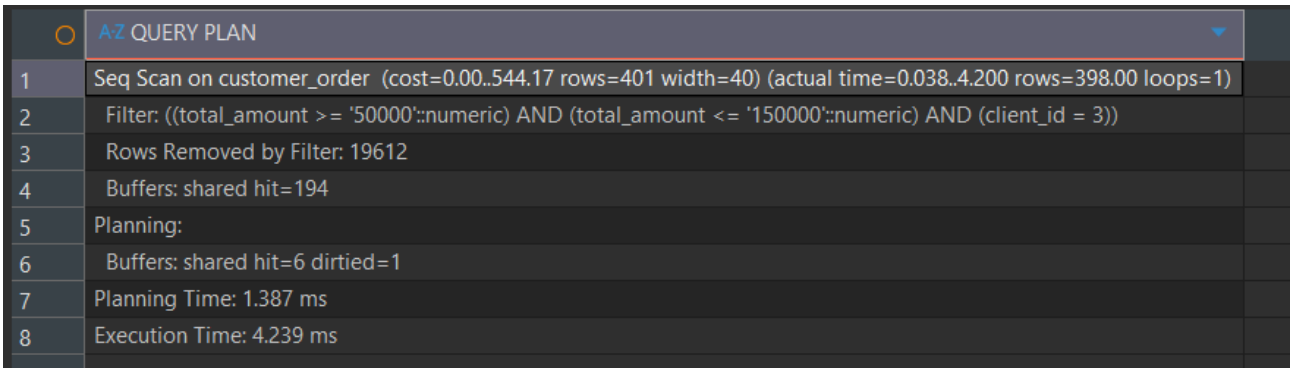


Рисунок 4. План выполнения Seq Scan с составным условием (до создания индекса)

Проводим анализ плана «ДО»:

— Узел плана: Seq Scan on customer_order — индекса, пригодного для этих условий, нет, поэтому читается вся таблица: 194 блока (столько же, сколько при остальных полных сканированиях этой таблицы, рисунки 8 и 16).

— Каждая из 20 010 строк проверяется по обоим условиям; Rows Removed by Filter: 19612; Planning Time: 1,387; Execution Time: 4,239.

Шаг 2. Создание индекса

Столбец с точным совпадением ставим первым, а столбец диапазона — вторым. Индекс отсортирован сначала по client_id, а внутри каждого клиента — по total_amount, поэтому PostgreSQL переходит к записям клиента №3 и забирает подряд значения от 50 000 до 150 000. Если поставить диапазон первым, индекс «сломается» уже на первом условии-диапазоне.

Листинг 6. Создание составного индекса

```
CREATE INDEX idx_customer_order_client_amount
ON customer_order (client_id, total_amount);
```

Шаг 3. Анализ «ПОСЛЕ» (с индексом)

Листинг 7. Повторный анализ запроса

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM customer_order
WHERE client_id = 3
  AND total_amount BETWEEN 50000 AND 150000;
```

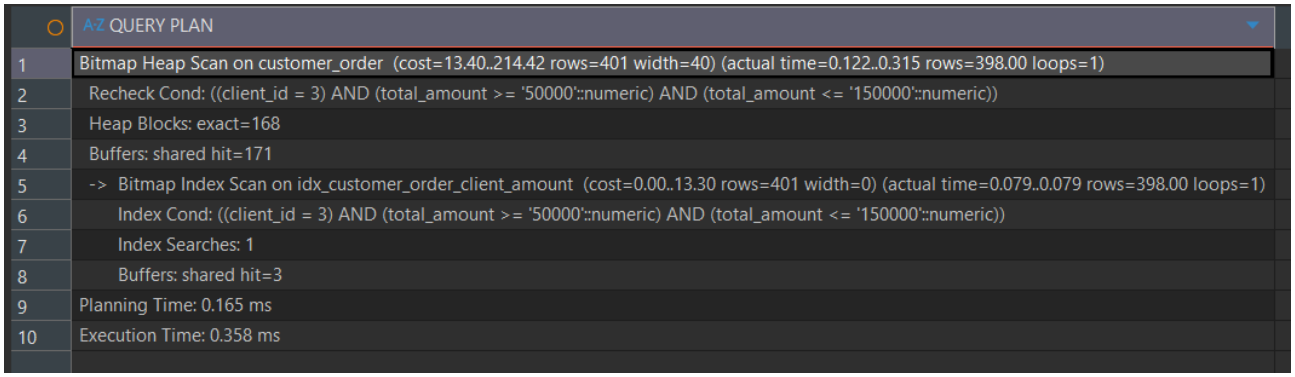


Рисунок 5. План выполнения Bitmap Index Scan по составному индексу

Проводим анализ плана «ПОСЛЕ»:

— Узел плана: **Bitmap Index Scan on idx_customer_order_client_amount** и **Bitmap Heap Scan**. Внутренний

узел находит по индексу TID подходящих строк (оценка rows=401) без построчной фильтрации, затем строится битовая карта страниц, и родительский узел читает только эти страницы таблицы.

— Буферы: для индекса прочитано 3 блока (shared read = 3), для таблицы — 172 блока (Heap Blocks: exact = 172); всего **175 блоков** против 194 при Seq Scan.

— Снижение числа блоков скромное, потому что ≈400 найденных строк разбросаны по таблице случайно (значения созданы функцией random()) и попадают в 172 из 194 страниц. Радикально сократить чтение таблицы можно физической сортировкой (CLUSTER, п. 3.2) или покрывающим индексом (п. 2.3).

— Planning Time: 0,165; Execution Time: 0,358.

Шаг 4. Сравнительная таблица

Таблица 2. Сравнение показателей запроса до и после создания составного индекса

Метрика	До оптимизации	После оптимизации	Вывод
Узел плана	Seq Scan	Bitmap Index Scan + Bitmap Heap Scan	Применён составной индекс вместо полного перебора
Порядок столбцов индекса	—	client_id, total_amount	Равенство стоит перед диапазоном: правило левого префикса соблюдено
Rows Removed by Filter	19612	0	Построчная фильтрация таблицы не нужна
Блоки индекса	—	3	Индекс читается за считанные блоки
Блоки таблицы	194	172	Строки клиента разбросаны по таблице случайно
Всего блоков	194	175	Небольшое снижение нагрузки на ввод-вывод
Planning Time	1,387	0,165	
Execution Time	4,239	0,358	

Вывод. Составной индекс (client_id, total_amount) позволяет сразу перейти к записям клиента и ограничиться диапазоном сумм. Выигрыш по блокам здесь невелик (194 → 175), так как нужные строки физически разбросаны по таблице: эффективность индекса определяется не только селективностью условия, но и порядком хранения данных.

1.3 Индекс по выражению (Function-based)

Задача. Реализовать регистронезависимый поиск услуги по названию: WHERE LOWER(name) = 'тур №10000'.

Шаг 1. Анализ «ДО» (без индекса по выражению)

Листинг 8. Анализ запроса с функцией LOWER()

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM service
WHERE LOWER(name) = 'тур №10000';
```

	QUERY PLAN
1	Seq Scan on service (cost=0.00..603.15 rows=100 width=95) (actual time=6.466..11.603 rows=1.00 loops=
2	Filter: (lower((name)::text) = 'тур №10000'::text)
3	Rows Removed by Filter: 20009
4	Buffers: shared hit=303
5	Planning:
6	Buffers: shared hit=17 dirtied=2
7	Planning Time: 2.540 ms
8	Execution Time: 11.629 ms

Рисунок 6. План выполнения Seq Scan при поиске с LOWER()

Проводим анализ плана «ДО»:

— Несмотря на существующий индекс idx_service_name (п. 1.1), выполняется **Seq Scan**: индекс хранит значения name, а не LOWER(name), поэтому условие с функцией он обслужить не может.

— Функция LOWER() вычисляется «на лету» для каждой из 20 010 строк; отброшено 20 009 строк, прочитано 303 блока.

— Execution Time = 11,629 мс — в ≈4,2 раза дольше, чем при поиске по тому же столбцу без функции (2,757 мс, п. 1.1) при том же числе блоков. Разница — это затраты процессора на вычисление функции; оценка cost выросла с 553.12 до 603.15.

— Оценка rows=100 не совпадает с фактическим результатом (1 строка): статистики по выражению LOWER(name) нет, и оптимизатор использует оценку по умолчанию.

Шаг 2. Создание индекса по выражению

В скобках передаём не имя столбца, а всё выражение целиком. Дерево индекса хранит уже вычисленные значения LOWER(name); при вставке новой строки значение вычисляется автоматически.

Листинг 9. Создание индекса по выражению

```
CREATE INDEX idx_service_name_lower
ON service (LOWER(name));
```

Шаг 3. Анализ «ПОСЛЕ» (с индексом)

Условие в запросе должно в точности совпадать с выражением в индексе.

Листинг 10. Повторный анализ запроса

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM service
WHERE LOWER(name) = 'тур №10000';
```

	QUERY PLAN
1	Bitmap Heap Scan on service (cost=5.06..220.95 rows=100 width=95) (actual time=0.260..0.261 rows=1.00 loc
2	Recheck Cond: (lower((name)::text) = 'тур №10000'::text)
3	Heap Blocks: exact=1
4	Buffers: shared hit=1 read=2
5	-> Bitmap Index Scan on idx_service_name_lower (cost=0.00..5.04 rows=100 width=0) (actual time=0.231..0.
6	Index Cond: (lower((name)::text) = 'тур №10000'::text)
7	Index Searches: 1
8	Buffers: shared read=2
9	Planning:
10	Buffers: shared hit=7 read=1
11	Planning Time: 2.470 ms
12	Execution Time: 0.290 ms

Рисунок 7. План выполнения Bitmap Index Scan по индексу по выражению

Проводим анализ плана «ПОСЛЕ»:

— Узел плана: **Bitmap Index Scan on idx_service_name_lower** и **Bitmap Heap Scan**. Значение ищется в индексе, где названия уже приведены к нижнему регистру.

— Вычисление LOWER() для каждой строки исчезло, Rows Removed by Filter отсутствует.

— Буферы: shared hit = 1, read = 2, всего **3 блока** (2 блока индекса и 1 блок таблицы, Heap Blocks: exact = 1) против 303.

— Оценка rows=100 осталась прежней: статистика по выражению собирается командой ANALYZE. Поэтому выбран Bitmap Scan, хотя найдена 1 строка; на результат это не влияет.

— Время: корневой узел плана выполнился за ≈0,063 мс (actual time) против 11,629 мс, то есть более чем в 100 раз быстрее; Planning Time и Execution Time: 2,470 и 0,290.

Шаг 4. Сравнительная таблица

Таблица 3. Сравнение показателей запроса до и после создания индекса по выражению

Метрика	До оптимизации	После оптимизации	Вывод
Узел плана	Seq Scan	Bitmap Index Scan + Bitmap Heap Scan	Индекс по выражению распознан оптимизатором
Вычисление LOWER()	Для всех 20 010 строк	Не выполняется на лету	С процессора снята вычислительная нагрузка
Rows Removed by Filter	20 009	0	Лишние строки не читаются
Всего блоков	303	3	Нагрузка на ввод-вывод снизилась в ≈100 раз
Planning Time	2,540 мс	2,470	
Execution Time	11,629 мс	0,290	Ускорение более чем в 100 раз

Вывод. Если в блоке WHERE к столбцу применяется функция, обычный индекс бесполезен. Индекс по выражению сократил число блоков с 303 до 3, а время выполнения — с 11,6 мс до долей миллисекунды.

1.4 Частичный (отфильтрованный) индекс

Задача. Найти неоплаченные заказы (paid_at IS NULL) в таблице customer_order.

Шаг 1. Анализ «ДО» (без индекса)

Листинг 11. Анализ запроса по узкому срезу данных

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM customer_order
WHERE paid_at IS NULL;
```

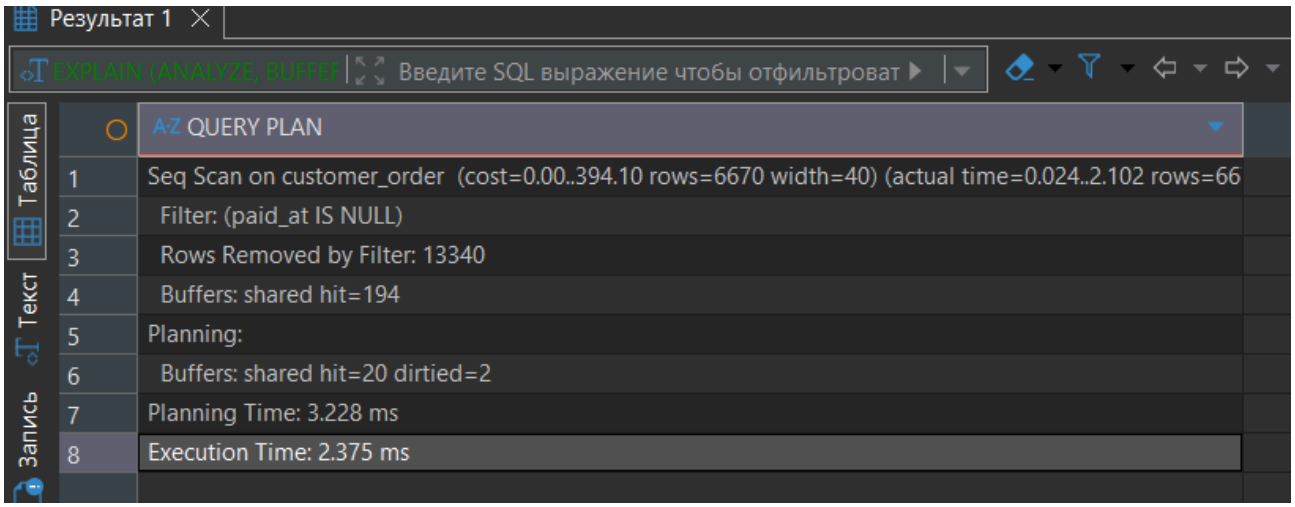


Рисунок 8. План выполнения Seq Scan при поиске неоплаченных заказов

Проводим анализ плана «ДО»:

— Узел плана: **Seq Scan on customer_order**; прочитано 194 блока (shared hit = 194).

— Отброшено 13 340 строк, возвращено 6 670 (20 010 − 13 340), то есть около 33 % таблицы.

— Planning Time = 3,228 мс, Execution Time = 2,375 мс.

Шаг 2. Создание частичного индекса

Условие WHERE записывается прямо в определении индекса: в него попадают только неоплаченные заказы, а оплаченные (две трети таблицы) в индекс не записываются.

Листинг 12. Создание частичного индекса

```
CREATE INDEX idx_customer_order_unpaid
ON customer_order (id)
WHERE paid_at IS NULL;
```

Шаг 3. Анализ «ПОСЛЕ» (с индексом)

Листинг 13. Повторный анализ запроса

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM customer_order
WHERE paid_at IS NULL;
```

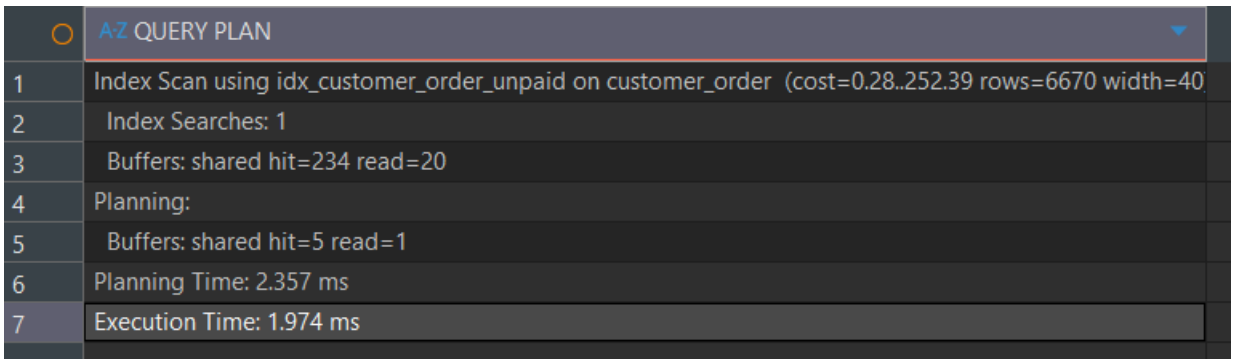


Рисунок 9. План выполнения Index Scan по частичному индексу

Проводим анализ плана «ПОСЛЕ»:

— Узел плана: **Index Scan using idx_customer_order_unpaid**. Условие запроса совпадает с условием частичного индекса, поэтому планировщик его использует; строка Rows Removed by Filter исчезла.

— Буферы: shared hit = 234, read = 20, всего **254 блока** — больше, чем при Seq Scan (194). Запрос возвращает 6 670 строк (≈33 % таблицы) и выбирает все столбцы (SELECT *), поэтому для каждой строки нужен переход из индекса в таблицу, а строки разбросаны по страницам.

— Время: Execution Time 1,974 мс против 2,375 мс (≈1,2 раза), Planning Time 2,357 мс против 3,228 мс; итого 4,331 мс против 5,603 мс.

— Размер индекса — 168 kB (рисунок 26): он содержит только 6 670 из 20 010 строк.

Шаг 4. Сравнительная таблица

Таблица 4. Сравнение показателей запроса до и после создания частичного индекса

Метрика	До оптимизации	После оптимизации	Вывод
Узел плана	Seq Scan	Index Scan	Применён частичный индекс
Возвращено строк	6 670	6 670	Результат запроса не изменился
Rows Removed by Filter	13 340	0	Лишняя фильтрация исчезла
Всего блоков (hit + read)	194	254 (234 + 20)	Чтения стало больше: выбирается треть таблицы, SELECT * требует обращения к таблице
Размер индекса	—	168 kB	Индексируются только неоплаченные заказы

Метрика	До оптимизации	После оптимизации	Вывод
Overhead при вставке оплаченных заказов	—	Отсутствует	Индекс обновляется только при добавлении неоплаченных заказов
Planning Time	3,228 мс	2,357 мс	Изменение на уровне погрешности
Execution Time	2,375 мс	1,974 мс	Ускорение в ≈1,2 раза
Итого (Planning + Execution)	5,603 мс	4,331 мс	Общий выигрыш ≈1,3 раза

Вывод. Частичный индекс эффективен для узкого среза данных (единицы процентов строк). Здесь неоплаченных заказов треть, поэтому выигрыш по времени небольшой, а по числу блоков индекс даже проигрывает полному сканированию. Сохраняется главное преимущество: оплаченные заказы (две трети вставок) в индекс не попадают и не замедляют запись.

Задание №2: специфические индексы и продвинутые техники

2.1 Индексирование сетевых адресов (Hash и GiST)

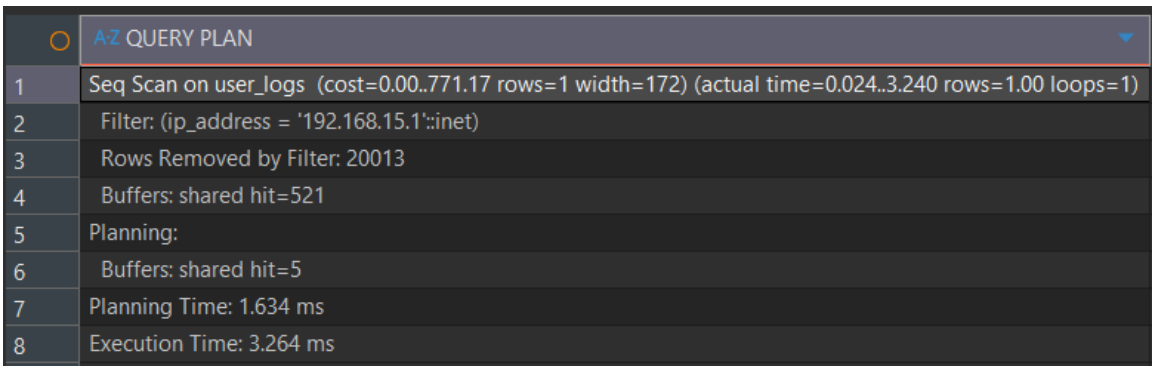
Задача. Сравнить производительность поиска по точному IP-адресу (Hash) и по маске подсети (GiST) в таблице user_logs, столбец ip_address имеет тип inet.

Часть 1. Точный поиск и Hash-индекс

Шаг 1. Анализ «ДО» (без индекса)

Листинг 14. Анализ запроса с точным поиском IP-адреса

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM user_logs
WHERE ip_address = '192.168.15.1'::inet;
```



	QUERY PLAN
1	Seq Scan on user_logs (cost=0.00..771.17 rows=1 width=172) (actual time=0.024..3.240 rows=1.00 loops=1)
2	Filter: (ip_address = '192.168.15.1'::inet)
3	Rows Removed by Filter: 20013
4	Buffers: shared hit=521
5	Planning:
6	Buffers: shared hit=5
7	Planning Time: 1.634 ms
8	Execution Time: 3.264 ms

Рисунок 10. План выполнения Seq Scan при точном поиске IP-адреса

Проводим анализ плана «ДО»:

— Узел плана: **Seq Scan on user_logs**; отброшено 20 013 строк ради одной записи.

— Буферы: shared hit = 521 (≈4,1 МБ). Таблица user_logs занимает больше блоков, чем service (303), так как строки шире из-за JSONB-документов (width = 172).

— Planning Time = 1,634 мс; Execution Time = 3,26 мс

Шаг 2. Создание Hash-индекса

Тип индекса нужно указать явно (USING hash), иначе PostgreSQL создаст B-Tree.

Листинг 15. Создание Hash-индекса

```
CREATE INDEX idx_user_logs_ip_hash
ON user_logs USING hash (ip_address);
```

Шаг 3. Анализ «ПОСЛЕ» (с индексом)

Листинг 16. Повторный анализ запроса

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM user_logs
WHERE ip_address = '192.168.15.1'::inet;
```

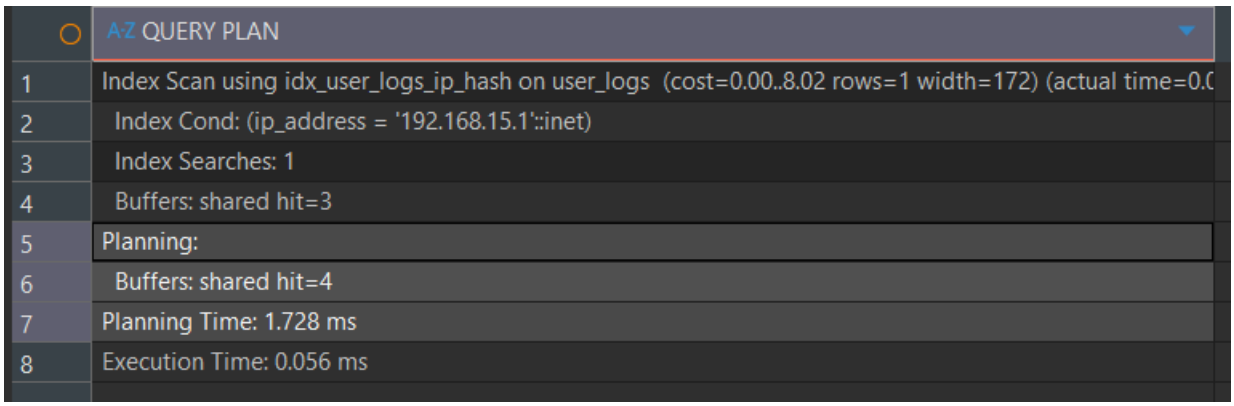


Рисунок 11. План выполнения Index Scan с использованием Hash-индекса

Проводим анализ плана «ПОСЛЕ»:

Оптимизатор применил **Index Scan using idx_user_logs_ip_hash**, так как условие использует оператор точного равенства (=). Хэш от адреса сразу указывает на нужную ячейку индекса: shared hit = 3 вместо 521, Execution Time = 0,056 мс, Planning Time = 1,728 мс. Запрос быстрее примерно в 70 раз (4,03 → 0,056 мс).

Часть 2. Поиск по маске подсети и GiST-индекс

Шаг 4. Анализ «ДО» (Hash-индекс существует)

Для аналитики безопасности требуется найти все записи из подсети 192.168.15.0/24 оператором включения «<<».

Листинг 17. Поиск записей по маске подсети

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM user_logs
WHERE ip_address << '192.168.15.0/24'::inet;
```

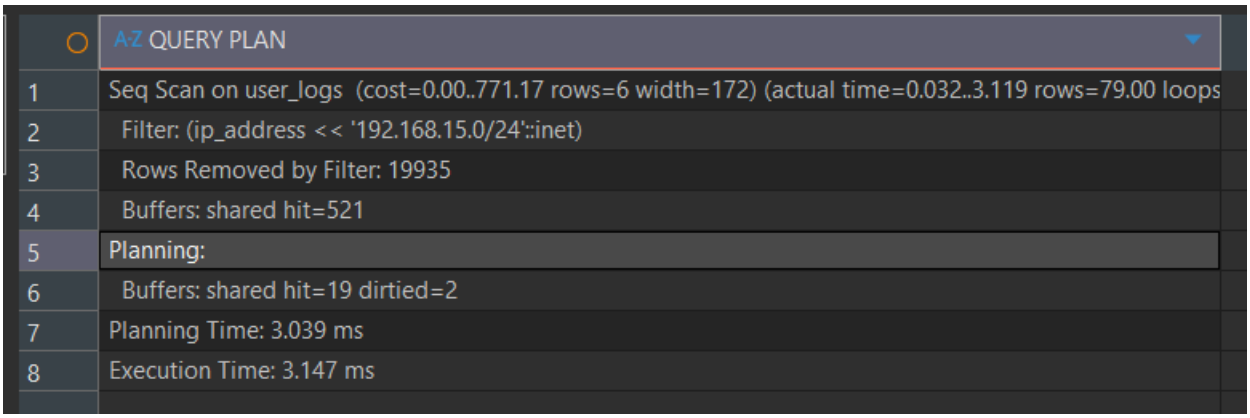


Рисунок 12. План выполнения Seq Scan при поиске по маске подсети (Hash-индекс не используется)

Проводим анализ плана:

— Hash-индекс поддерживает только оператор равенства, поэтому для оператора << он игнорируется и выполняется **Seq Scan**: отброшено 19 935 строк, найдено 79, прочитано 521 блок.

— Planning Time = 3,039 мс, Execution Time = 3,147 мс.

Шаг 5. Создание GiST-индекса

Для типа inet в GiST необходимо явно указать класс операторов inet_ops.

Листинг 18. Создание GiST-индекса

```
CREATE INDEX idx_user_logs_ip_gist
ON user_logs USING gist (ip_address inet_ops);
```

Шаг 6. Анализ «ПОСЛЕ» (с индексом)

Листинг 19. Повторный анализ запроса

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM user_logs
WHERE ip_address << '192.168.15.0/24'::inet;
```

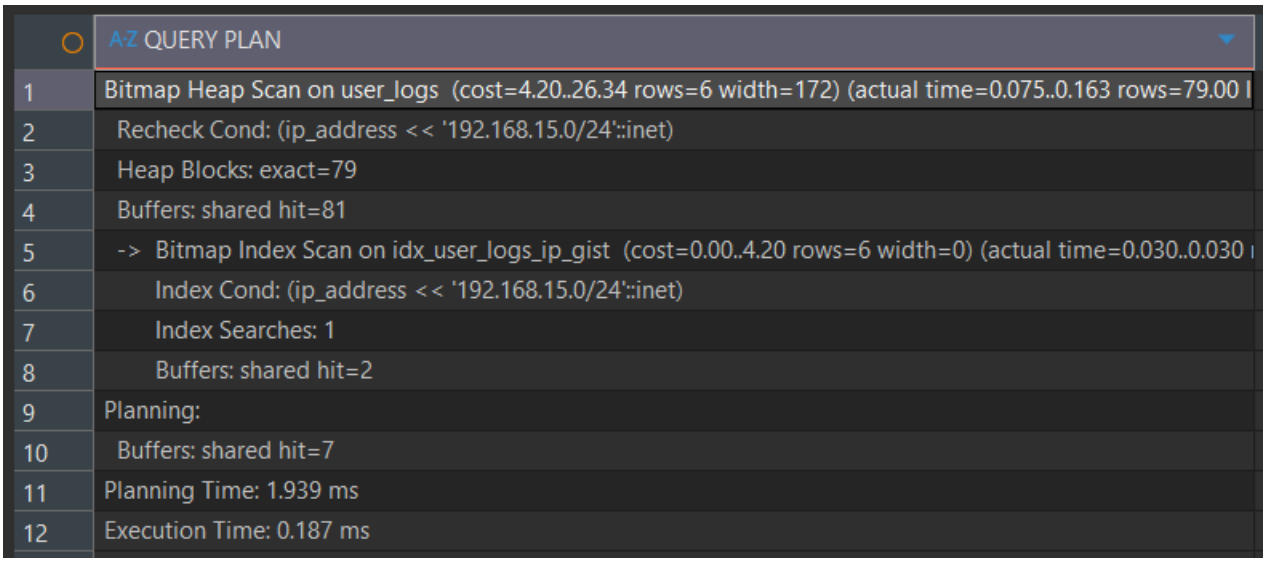


Рисунок 13. План выполнения Bitmap Index Scan с использованием GiST-индекса

Проводим анализ плана «ПОСЛЕ»:

— Узел плана: **Bitmap Index Scan on idx_user_logs_ip_gist** и **Bitmap Heap Scan**. Вхождение адреса в подсеть проверяется прямо в структуре индекса.

— Буферы: индекс прочитан за 2 блока, таблица — за 79 (Heap Blocks: exact = 79), всего **81 блок** против 521 (меньше в ≈6,4 раза). Блоков

таблицы столько же, сколько найденных строк: каждая подходящая запись лежит на своей странице.

— Корневой узел плана выполнился за ≈0,107 мс (actual time) против 3,147 мс, то есть примерно в 30 раз быстрее; Planning Time и Execution Time:1,939 и 0,187.

— Оценка rows=6 заметно ниже фактических 79 строк: статистика по подсетям неточна, но на выбор индекса это не повлияло.

Шаг 7. Сравнительная таблица

Таблица 5. Сравнение методов доступа для поиска по IP-адресу

Сценарий	Индекс	План	Блоков	Execution Time
ip_address = '192.168.15.1'	Нет	Seq Scan	521	3,264мс
ip_address = '192.168.15.1'	Hash	Index Scan	3	0,056 мс
ip_address << '192.168.15.0/24'	Hash (не применяется)	Seq Scan	521	3,147 мс
ip_address << '192.168.15.0/24'	GiST (inet_ops)	Bitmap Index Scan + Bitmap Heap Scan	81	0,187 мс

Вывод. Для точного равенства подходит Hash-индекс (поиск за O(1), 3 блока), для поиска по маске подсети — GiST: B-Tree и Hash логику вхождения в подсеть не понимают. По данным pg_stat_user_indexes (рисунок 26) Hash-индекс занимает 1040 kB, а GiST — 528 kB, то есть на этой выборке Hash не оказался компактнее GiST.

2.2 Индексирование временных рядов (BRIN)

Задача. Сравнить физические размеры B-Tree и BRIN индексов по столбцу created_at таблицы customer_order и проверить работу BRIN на хаотично расположенных и на упорядоченных данных.

Шаг 1. Создание и замер B-Tree

Размер индекса в байтах возвращает функция pg_relation_size, а pg_size_pretty приводит его к удобному виду (kB, MB).

Листинг 20. Создание B-Tree индекса и замер его размера

```
CREATE INDEX idx_customer_order_created_btree
ON customer_order (created_at);

SELECT pg_size_pretty(
    pg_relation_size('idx_customer_order_created_btree')
) AS btree_size;
```

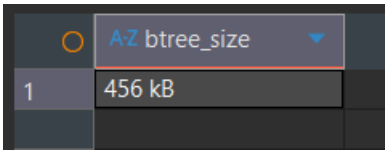


Рисунок 14. Замер физического размера B-Tree индекса

B-Tree хранит ссылку на каждую из 20 010 строк, поэтому занимает **456 kB**. На таблице в миллионы строк такой индекс разрастается до гигабайтов.

Шаг 2. Создание и замер BRIN

BRIN хранит не ссылки на строки, а минимальное и максимальное значение столбца для группы страниц. Размер таблицы небольшой (194 страницы), поэтому при стандартных 128 страницах на диапазон получилось бы всего две записи; параметром pages_per_range = 1 заставляем BRIN строить отдельную сводку для каждой страницы. Предварительно удаляем B-Tree индекс.

Листинг 21. Создание BRIN-индекса и замер его размера

```
DROP INDEX IF EXISTS idx_customer_order_created_btree;

CREATE INDEX idx_customer_order_created_brin
ON customer_order USING brin (created_at)
WITH (pages_per_range = 1);

SELECT pg_size_pretty(
    pg_relation_size('idx_customer_order_created_brin')
) AS brin_size;
```

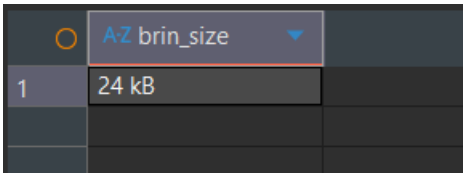


Рисунок 15. Замер физического размера BRIN-индекса

Размер BRIN составил **24 kB** — в 19 раз меньше, чем у B-Tree (456 kB).

Шаг 3. Диапазонная выборка на хаотичных данных

Листинг 22. Диапазонная выборка по date

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM customer_order
WHERE created_at BETWEEN NOW() - INTERVAL '180 days'
                     AND NOW() - INTERVAL '90 days';
```

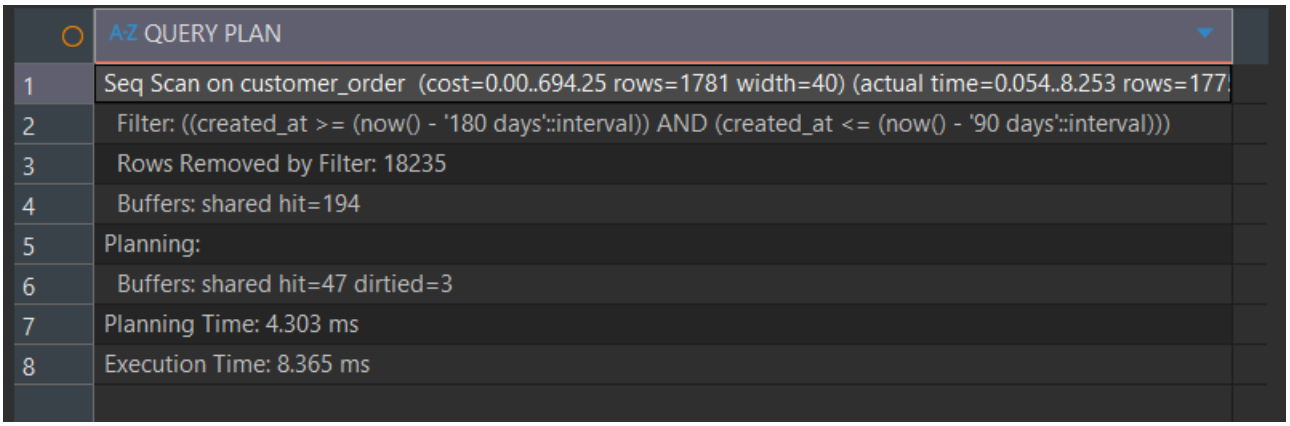


Рисунок 16. План выполнения Seq Scan при BRIN-индексе на хаотичных данных

Проводим анализ:

— Несмотря на наличие BRIN-индекса, выполнен **Seq Scan**: отброшено 18 235 строк, возвращено 1 775, прочитано 194 блока; Planning Time = 4,303 мс, Execution Time = 8,365 мс.

— Причина: значения created_at сгенерированы функцией random(), поэтому на каждой странице есть строки с самыми разными датами. Диапазоны min/max пересекаются на всех страницах, и BRIN не может пропустить ни одну из них.

Шаг 4. Физическая кластеризация (CLUSTER)

BRIN эффективен, только если данные на диске лежат в естественном порядке. Чтобы создать такие условия, физически пересортируем таблицу командой CLUSTER по временному индексу, затем удаляем его и обновляем статистику.

Листинг 23. Физическая пересортировка таблицы и повторный анализ

```
CREATE INDEX tmp_customer_order_created_sort
ON customer_order (created_at);

CLUSTER customer_order USING tmp_customer_order_created_sort;

DROP INDEX tmp_customer_order_created_sort;

ANALYZE customer_order;

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM customer_order
WHERE created_at BETWEEN NOW() - INTERVAL '180 days'
                     AND NOW() - INTERVAL '90 days';
```

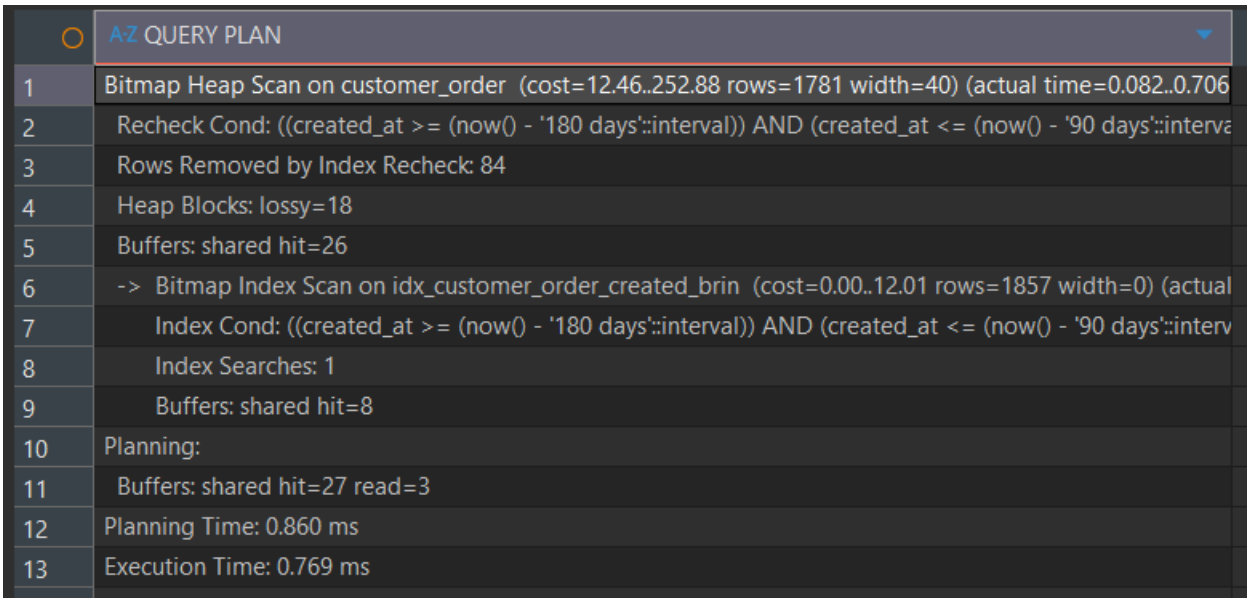


Рисунок 17. План выполнения Bitmap Index Scan по BRIN-индексу после физической кластеризации

Проводим анализ:

— Узел плана: **Bitmap Index Scan on idx_customer_order_created_brin** и **Bitmap Heap Scan**. BRIN отобрал только те группы страниц, где могут лежать даты запрошенного диапазона (Heap Blocks: lossy = 18).

— Rows Removed by Index Recheck: 84 — строки из отобранных страниц, не прошедшие повторную проверку условия.

— Буферы: shared hit = 26 (8 блоков индекса и 18 блоков таблицы) вместо 194 — меньше в ≈7,5 раза. Execution Time = 0,769 мс против 8,365 мс (быстрее в ≈11 раз), Planning Time = 0,860 мс.

Шаг 5. Сравнительные таблицы

Таблица 6. Сравнение B-Tree и BRIN индексов

Свойство индекса	B-Tree	BRIN	Вывод
Размер на 20 010 строк	456 kB	24 kB	BRIN меньше в 19 раз
Что хранится в индексе	Ссылка (TID) на каждую строку	Min и max значения для группы страниц	BRIN хранит метаданные блоков, а не строк
Рост размера	Пропорционально числу строк	Пропорционально числу страниц	На больших таблицах разница достигает тысяч раз
Условие эффективности	Не зависит от порядка строк на диске	Данные должны лежать на диске упорядоченно	На хаотичных данных BRIN не используется (Seq Scan)

Таблица 7. Результаты запроса с BRIN-индексом до и после физической кластеризации

Метрика	До CLUSTER	После CLUSTER	Вывод
Узел плана	Seq Scan	Bitmap Index Scan (BRIN) + Bitmap Heap Scan	BRIN начал использоваться
Отброшено строк	18 235 (Rows Removed by Filter)	84 (Rows Removed by Index Recheck)	Лишних строк стало в ≈200 раз меньше
Всего блоков	194	26 (8 + 18)	Нагрузка на ввод-вывод снизилась в ≈7,5 раза
Planning Time	4,303 мс	0,860 мс	
Execution Time	8,365 мс	0,769 мс	Выполнение ускорилось в ≈11 раз
Итого (Planning + Execution)	12,668 мс	1,629 мс	Общий выигрыш ≈7,8 раза

Вывод. BRIN занимает в 19 раз меньше места, чем B-Tree, но работает только при упорядоченном физическом хранении данных: на хаотичных данных он не использовался, а после CLUSTER число читаемых блоков снизилось с 194 до 26. Для реальных журналов, где записи добавляются в порядке времени, такая упорядоченность возникает естественно.

2.3 Покрывающий индекс (Index-Only Scan)

Задача. Получать тип услуги и страну по названию услуги, не обращаясь к таблице service: SELECT service_type, country_id ... WHERE name = 'Typ №5555'.

Шаг 1. Анализ «ДО» (обычный индекс)

Листинг 24. Анализ запроса по обычному индексу

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT service_type, country_id
FROM service
WHERE name = 'Typ №5555';
```

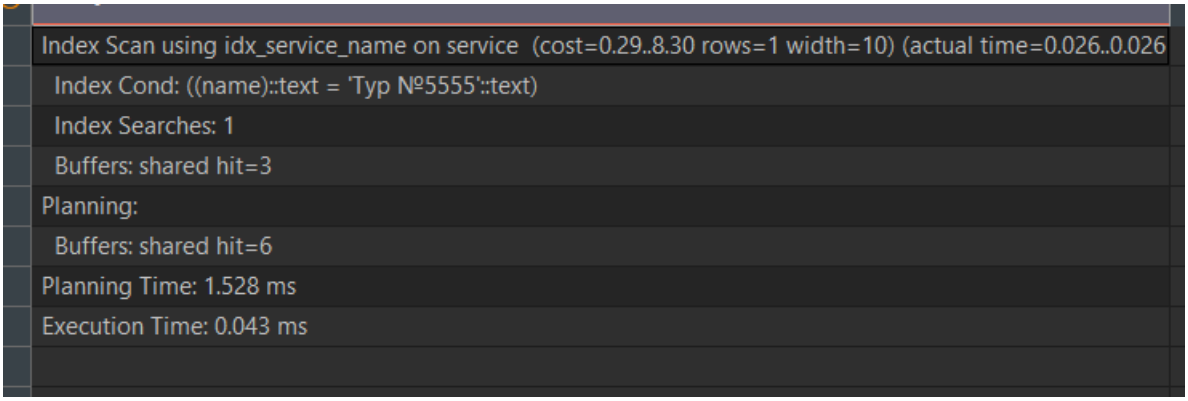


Рисунок 18. План выполнения Index Scan по обычному индексу idx_service_name

Проводим анализ плана «ДО»:

— Узел плана: **Index Scan using idx_service_name** (cost 0.29..8.30). Индекс содержит только name, поэтому за значениями service_type и country_id после поиска в дереве приходится обращаться к таблице (Heap Fetch).

— Буферы: shared hit = 3; Planning Time = 1,528 мс; Execution Time = 0,043 мс.

Шаг 2. Создание покрывающего индекса

Дерево строится по name, а директива INCLUDE прикрепляет service_type и country_id к листьям индекса, не участвуя в сортировке. Команда VACUUM обновляет карту видимости (Visibility Map): без неё Index Only Scan вынужден проверять видимость строк в таблице (Heap Fetches > 0). Обычный индекс idx_service_name удаляется, чтобы оптимизатор использовал покрывающий.

Листинг 25. Создание покрывающего индекса

```
CREATE INDEX idx_service_name_cover
ON service (name)
INCLUDE (service_type, country_id);

VACUUM (ANALYZE) service;

DROP INDEX IF EXISTS idx_service_name;
```

Шаг 3. Анализ «ПОСЛЕ» (Index-Only Scan)

Запрашиваются строго те поля, которые есть в индексе или в его блоке INCLUDE: при SELECT * преимущество пропало бы.

Листинг 26. Повторный анализ запроса

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT service_type, country_id
FROM service
WHERE name = 'Typ №5555';
```

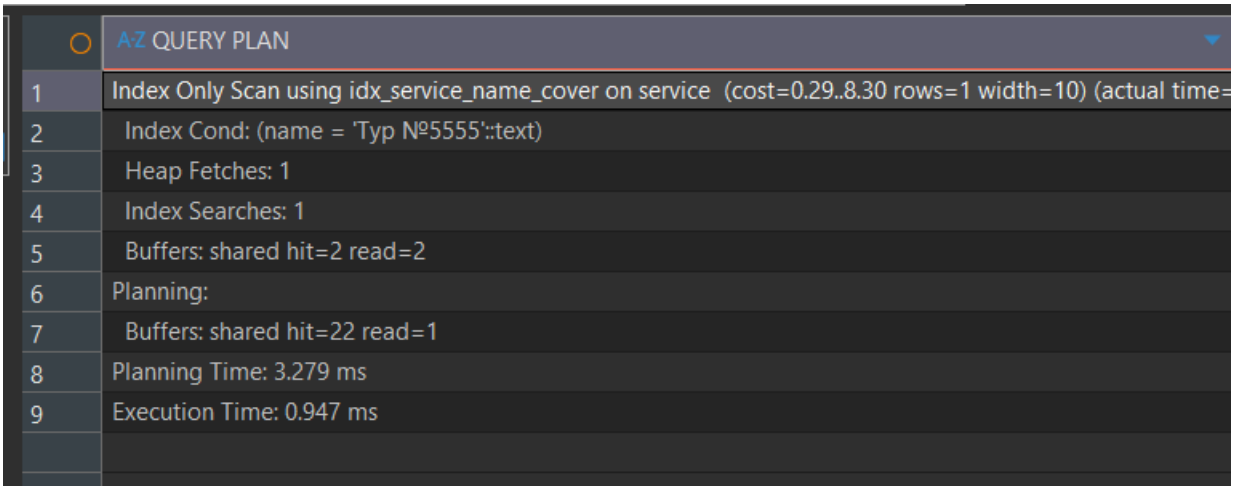


Рисунок 19. План выполнения Index-Only Scan

Проводим анализ плана «ПОСЛЕ»:

— Узел плана: Index Only Scan using idx_service_name_cover — все запрошенные поля находятся в листьях индекса.

— Heap Fetches: 1.

— Буферы: 2 против 3; Planning Time: 3,279; Execution Time: 0,947.

— Цена решения: размер покрывающего индекса — 992 kB (рисунок 26), больше, чем у индекса по LOWER(name) на том же столбце (632 kB), так как в листьях хранятся ещё service_type и country_id; кроме того, при изменении этих столбцов обновляется и индекс.

Шаг 4. Сравнительная таблица

Таблица 8. Сравнение обычного индекса и покрывающего индекса

Метрика	Index Scan (обычный индекс)	Index Only Scan (покрывающий индекс)	Вывод
Узел плана	Index Scan	Index Only Scan	Таблица не читается
Чтение таблицы (Heap Fetch)	Требуется	1	INCLUDE исключает обращение к диску

Метрика	Index Scan (обычный индекс)	Index Only Scan (покрывающий индекс)	Вывод
Всего блоков	3	2	
Planning Time	1,528 мс	3,279	
Execution Time	0,043 мс	0,947	
Размер индекса	—	992 kB	Платой за скорость становится размер индекса

Вывод. Покрывающий индекс отдаёт ответ целиком из индекса и не обращается к таблице, но увеличивает размер индекса и замедляет INSERT/UPDATE, поэтому применяется точечно — для самых частых и узких запросов.

2.4 Работа с неструктурированными данными: JSONB и GIN-индекс

Задача. Найти в таблице user_logs записи, у которых в массиве tags внутри JSONB-столбца event_data присутствует тег «promo» (оператор включения @>).

Шаг 1. Поиск по JSONB без индекса

Листинг 27. Поиск по JSONB без использования GIN

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT log_id, user_id, created_at
FROM user_logs
WHERE event_data @> '{"tags": ["promo"]}'::jsonb;
```

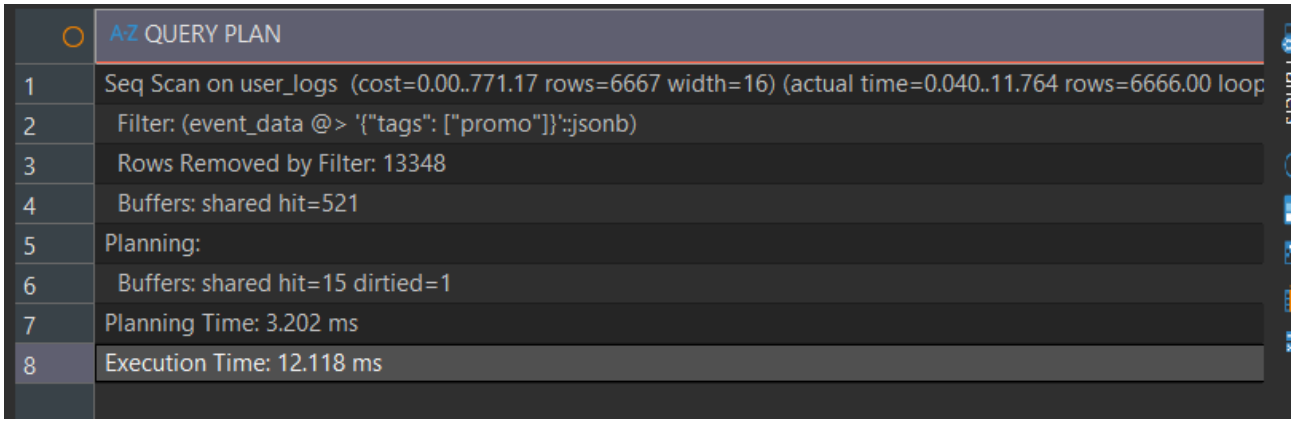


Рисунок 20. План выполнения Seq Scan при поиске по JSONB

Проводим анализ плана «ДО»:

— Узел плана: **Seq Scan on user_logs**. B-Tree индекс здесь бессилен: он индексирует значение столбца целиком, а не элементы внутри документа.

— Прочитаны все строки (521 блок), каждый JSONB-документ разбирается «на лету»; отброшено 13 348 строк, найдено 6 666.

— Planning Time = 3,202 мс, Execution Time = 12,118 мс.

Шаг 2. Создание инвертированного индекса (GIN)

Листинг 28. Создание GIN-индекса

```
CREATE INDEX idx_user_logs_event_gin
ON user_logs USING gin (event_data);
```

Шаг 3. Проверка эффективности GIN-индекса

Листинг 29. Повторный анализ запроса

```
EXPLAIN (ANALYZE, BUFFERS)
SELECT log_id, user_id, created_at
FROM user_logs
WHERE event_data @> '{"tags": ["promo"]}'::jsonb;
```

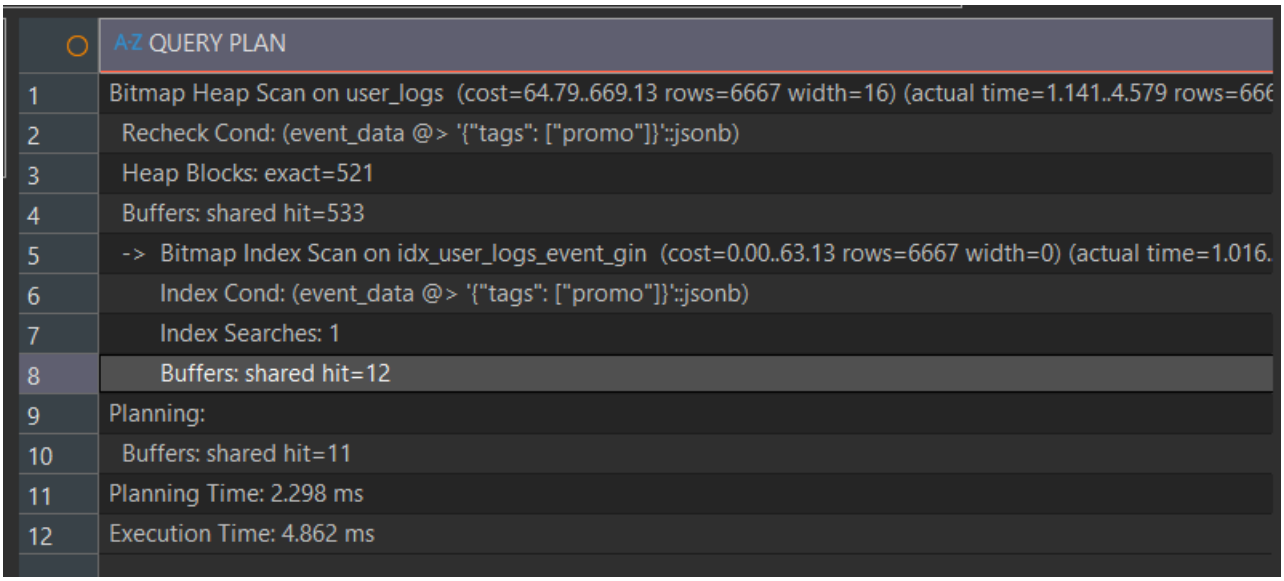


Рисунок 21. План выполнения Bitmap Index Scan с GIN-индексом

Проводим анализ плана «ПОСЛЕ»:

— Узел плана: **Bitmap Index Scan on idx_user_logs_event_gin** и **Bitmap Heap Scan**. GIN нашёл идентификаторы строк, где встречается «promo», за 12 блоков индекса.

— Буферы: shared hit = 533 (12 блоков индекса и 521 блок таблицы, Heap Blocks: exact = 521) — не меньше, чем при Seq Scan (521). Тег «promo» есть у трети записей, и они равномерно распределены по страницам, поэтому таблицу всё равно приходится читать почти целиком.

— Время: Execution Time = 4,862 мс против 12,118 мс (быстрее в ≈2,5 раза), Planning Time = 2,298 мс. Выигрыш получен за счёт того, что документы без искомого тега больше не разбираются.

— Размер индекса — 2936 kB (рисунок 26), то есть около 70 % размера самой таблицы (521 блок ≈ 4,1 МБ): GIN — тяжёлая структура.

Шаг 4. Сравнительная таблица

Таблица 9. Сравнение показателей запроса по JSONB до и после создания GIN-индекса

Метрика	До оптимизации	После оптимизации	Вывод
Узел плана	Seq Scan	Bitmap Index Scan + Bitmap Heap Scan	GIN заглядывает внутрь JSONB-документа
Rows Removed by Filter	13 348	0	Документы без тега не разбираются
Всего блоков	521	533 (12 + 521)	Блоки таблицы читаются все: «promo» у трети записей
Planning Time	3,202 мс	2,298 мс	Изменение на уровне погрешности
Execution Time	12,118 мс	4,862 мс	Ускорение в ≈2,5 раза
Размер индекса	—	2936 kB	≈70 % размера таблицы

Вывод. GIN ускорил поиск по JSONB в ≈2,5 раза, но число блоков не уменьшилось, потому что условию соответствует треть таблицы. Максимальный эффект GIN даёт для редких значений ключей и на таблицах в миллионы строк, где Seq Scan занимает секунды.

Задание №3: обслуживание базы данных

3.1 Влияние статистики (ANALYZE)

Задача. Смоделировать деградацию плана выполнения после массового обновления данных и исправить её актуализацией статистики. Планировщик берёт оценки числа строк из системного каталога pg_statistic (списки частых значений и гистограммы); после массовых изменений данных они перестают соответствовать действительности.

Шаг 1. Массовое обновление и анализ «сломанной» логики

Моделируем смену страны у части услуг и сразу анализируем запрос, не собирая статистику.

Листинг 30. Искажение статистики массовым обновлением

```
UPDATE service
SET country_id = 2
WHERE country_id = 1;

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM service
WHERE country_id = 1;
```

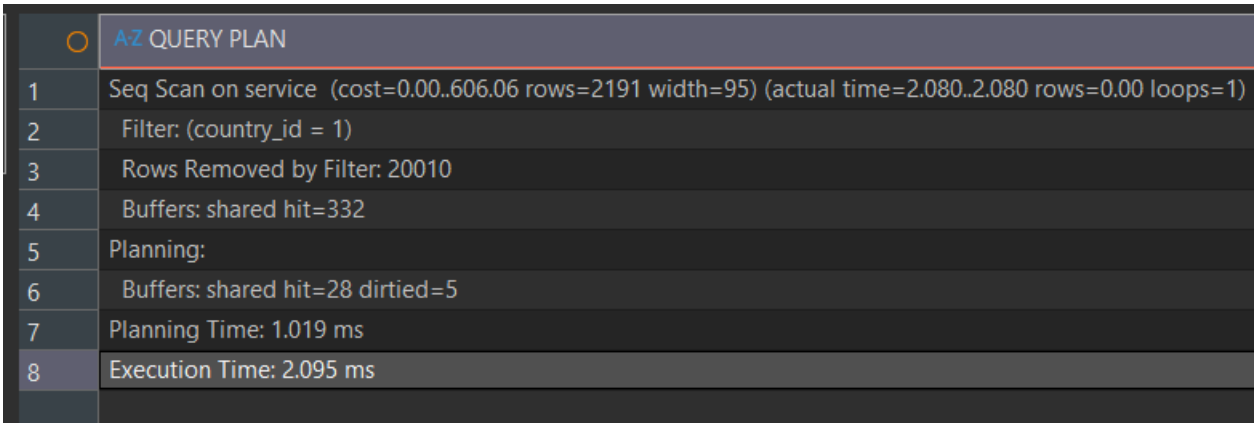


Рисунок 22. Деградация плана выполнения из-за устаревшей статистики

Проводим анализ:

— Планировщик ожидал найти **2191 строку** (rows=2191, по старой статистике около 11 % услуг имели country_id = 1), а нашёл **0** (actual rows=0): после UPDATE таких строк не осталось. Оценка стоимости (cost 606.06) завышена.

— Размер таблицы вырос с 303 до 332 блоков (shared hit = 332): UPDATE создал новые версии изменённых строк, а старые остались в таблице как «мёртвые» до очистки командой VACUUM (механизм MVCC).

Шаг 2. Обновление статистики (ANALYZE)

Листинг 31. Актуализация статистики и повторный анализ запроса

```
ANALYZE service;

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM service
WHERE country_id = 1;
```

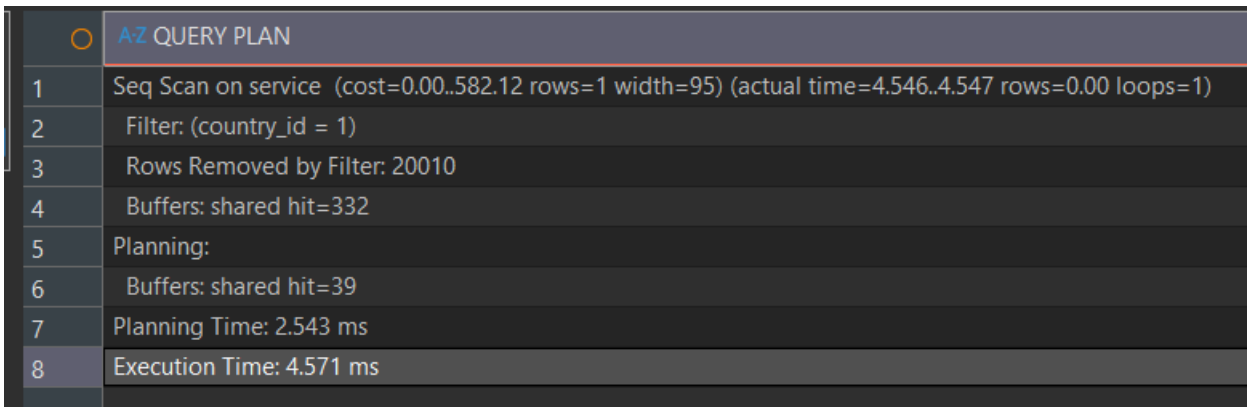


Рисунок 23. План выполнения после обновления статистики

Проводим анализ:

— Ожидание планировщика (rows=1) теперь практически совпадает с реальностью (0 строк), cost снизился с 606.06 до 582.12.

— План остался Seq Scan, потому что индекса по country_id нет; при его наличии точная оценка позволила бы оптимизатору выбрать Index Scan. Разброс времени между запусками (2,095 и 4,571 мс) не связан с ANALYZE: команда не меняет данные, а лишь даёт корректные оценки.

Шаг 3. Сравнительная таблица

Таблица 10. Сравнение плана запроса до и после обновления статистики

Метрика	До ANALYZE	После ANALYZE	Вывод
Ожидаемое число строк (rows)	2191	1	Оценка приведена в соответствие с данными
Фактическое число строк	0	0	Результат запроса не изменился
Cost	606.06	582.12	Стоимость плана оценивается точнее
Rows Removed by Filter	20 010	20 010	Индекса по country_id нет, поэтому Seq Scan сохранился
Всего блоков	332	332	Чтение таблицы не изменилось
Planning Time	1,019 мс	2,543 мс	Разброс времени не связан с ANALYZE
Execution Time	2,095 мс	4,571 мс	Разброс времени не связан с ANALYZE

Вывод. Устаревшая статистика ведёт к неверным оценкам и неоптимальным планам. Фоновый процесс autovacuum запускает ANALYZE автоматически, но после разовых массовых операций (миграции, массовые

UPDATE и DELETE) он может не успеть, поэтому статистику в таких случаях обновляют вручную сразу после операции.

3.2 Физическая сортировка данных (CLUSTER)

Задача. Показать влияние физического порядка строк на диапазонный запрос по created_at (таблица customer_order). К этому пункту таблица уже была один раз отсортирована по created_at командой CLUSTER в п. 2.2 (листинг 23), поэтому для оценки эффекта сопоставляются результаты обоих пунктов.

Шаг 1. Анализ «ДО» повторной кластеризации

Листинг 32. Создание индекса и замер обращений к буферам

```
CREATE INDEX idx_customer_order_created_cluster
ON customer_order (created_at);

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM customer_order
WHERE created_at BETWEEN NOW() - INTERVAL '180 days'
                     AND NOW() - INTERVAL '90 days';
```

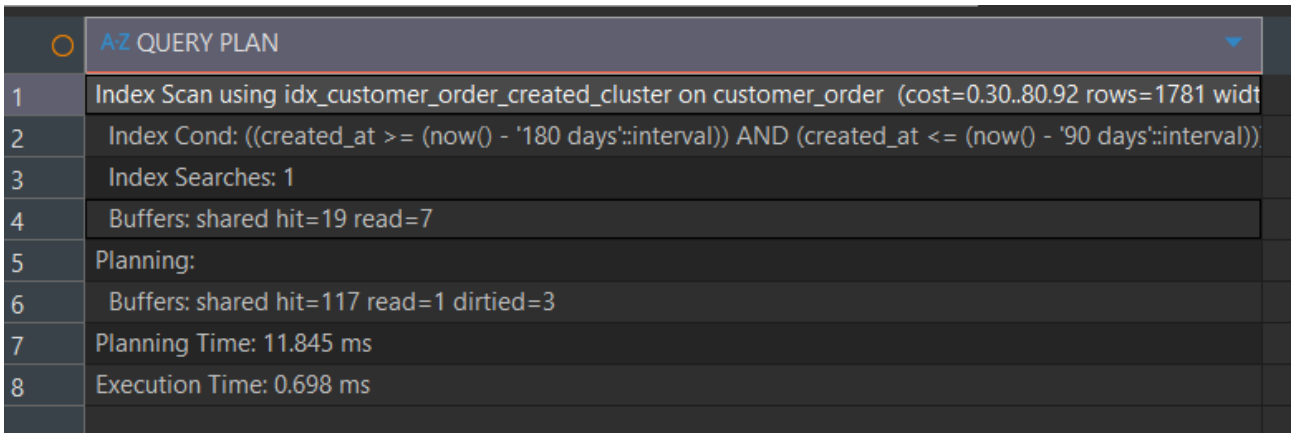


Рисунок 24. План выполнения диапазонного запроса до повторной кластеризации

Проводим анализ:

— Узел плана: **Index Scan using idx_customer_order_created_cluster**. Для ≈1 800 строк диапазона прочитано всего 26 блоков (hit = 19, read = 7): после CLUSTER из п. 2.2 строки с соседними датами лежат в соседних страницах.

— Planning Time = 11,845 мс (после создания индекса планировщик загружает его метаданные), Execution Time = 0,698 мс.

Шаг 2. Выполнение системной кластеризации

Даём команду физически пересобрать таблицу по индексу.

Листинг 33. Физическая сортировка таблицы и повторный анализ

```
CLUSTER customer_order
USING idx_customer_order_created_cluster;

EXPLAIN (ANALYZE, BUFFERS)
SELECT *
FROM customer_order
WHERE created_at BETWEEN NOW() - INTERVAL '180 days'
                     AND NOW() - INTERVAL '90 days';
```

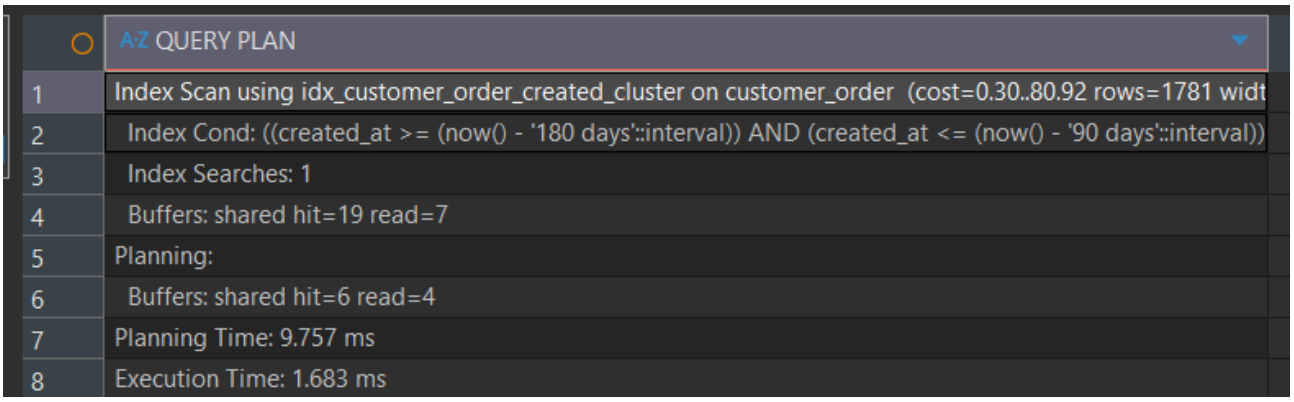


Рисунок 25. План выполнения диапазонного запроса после повторной кластеризации

Проводим анализ:

— План и число блоков не изменились: Index Scan, 26 блоков (hit = 19, read = 7). Повторный CLUSTER уже отсортированной таблицы дополнительного эффекта не даёт, а различие времени (0,698 и 1,683 мс) связано с состоянием кэша и нагрузкой, а не с порядком строк.

— Эффект кластеризации виден при сопоставлении с хаотичной таблицей из п. 2.2 (рисунок 16): там тот же запрос читал 194 блока и выполнялся Seq Scan за 8,365 мс.

Шаг 3. Сравнительная таблица

Таблица 11. Влияние физической сортировки на диапазонный запрос по created_at

Метрика	Хаотичная таблица (п. 2.2, до CLUSTER)	После CLUSTER (рисунок 24)	Повторный CLUSTER (рисунок 25)
Узел плана	Seq Scan	Index Scan	Index Scan
Всего блоков	194	26	26
Planning Time	4,303 мс	11,845 мс	9,757 мс
Execution Time	8,365 мс	0,698 мс	1,683 мс

Вывод. Физическая сортировка сократила чтение таблицы со 194 до 26 блоков (≈7,5 раза) и время выполнения с 8,4 до 0,7–1,7 мс (в первом случае оптимизатор был вынужден использовать Seq Scan, во втором — индексный доступ). CLUSTER требует эксклюзивной блокировки таблицы (ACCESS EXCLUSIVE LOCK), полностью переписывает таблицу и перестраивает все её индексы, а порядок строк не поддерживается автоматически и нарушается новыми INSERT и UPDATE. Поэтому команду выполняют в окна обслуживания и повторяют по мере необходимости.

3.3 Поиск неиспользуемых индексов

Задача. Провести аудит индексов, выявить неиспользуемые и удалить их. PostgreSQL ведёт статистику обращений к индексам, она доступна в представлении pg_stat_user_indexes (столбец idx_scan).

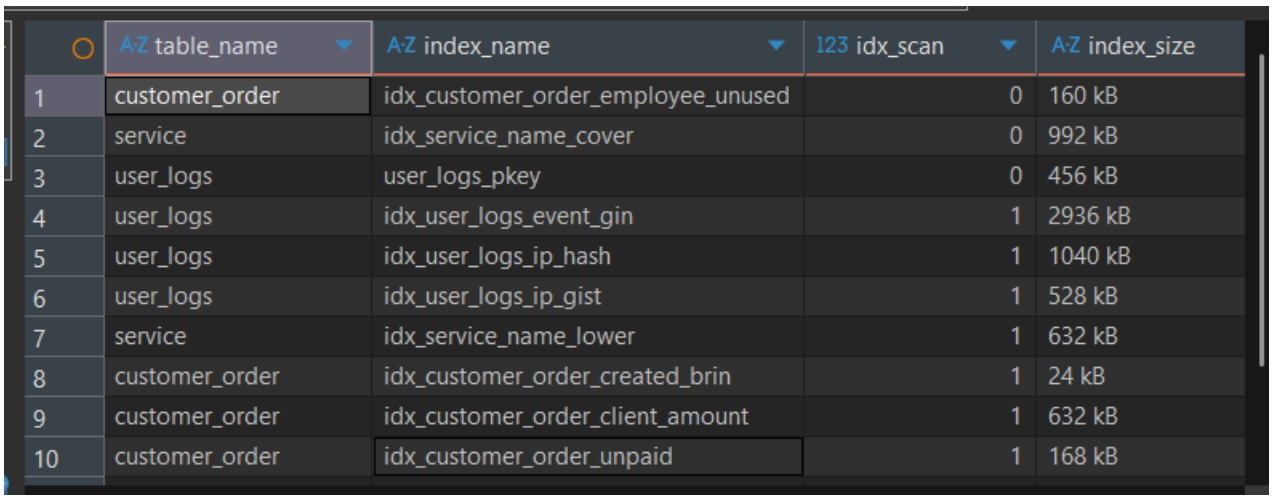
Шаг 1. Мониторинг статистики обращений к индексам

Для демонстрации создаём заведомо ненужный индекс по employee_id, после чего запрашиваем статистику по таблицам service, customer_order и user_logs.

Листинг 34. Создание индекса и запрос к системному представлению

```
CREATE INDEX idx_customer_order_employee_unused
ON customer_order (employee_id);

SELECT
    relname AS table_name,
    indexrelname AS index_name,
    idx_scan,
    pg_size_pretty(pg_relation_size(indexrelid)) AS index_size
FROM pg_stat_user_indexes
WHERE relname IN ('service', 'customer_order', 'user_logs')
ORDER BY idx_scan ASC;
```



	table_name	index_name	idx_scan	index_size
1	customer_order	idx_customer_order_employee_unused	0	160 kB
2	service	idx_service_name_cover	0	992 kB
3	user_logs	user_logs_pkey	0	456 kB
4	user_logs	idx_user_logs_event_gin	1	2936 kB
5	user_logs	idx_user_logs_ip_hash	1	1040 kB
6	user_logs	idx_user_logs_ip_gist	1	528 kB
7	service	idx_service_name_lower	1	632 kB
8	customer_order	idx_customer_order_created_brin	1	24 kB
9	customer_order	idx_customer_order_client_amount	1	632 kB
10	customer_order	idx_customer_order_unpaid	1	168 kB

Рисунок 26. Статистика использования индексов (pg_stat_user_indexes)

Проводим анализ:

Значение idx_scan = 0 имеют три индекса, но удалять можно не все:

Таблица 12. Индексы с нулевым числом обращений

Индекс	idx_scan	Размер	Решение
idx_customer_order_employee_unused	0	160 kB	Удалить: создан для демонстрации, ни один запрос его не использует
user_logs_pkey	0	456 kB	Оставить: индекс первичного ключа (имя оканчивается на pkey), необходим всегда
idx_service_name_cover	0	992 kB	Оставить: создан для п. 2.3; на момент замера ни один запрос к нему не обращался

Остальные индексы, созданные в ходе работы, использованы по одному разу (idx_scan = 1) — каждый в своём сценарии. Самый «тяжёлый» из них — GIN-индекс по event_data (2936 kB). В реальном проекте решение принимают после наблюдения за статистикой хотя бы неделю после релиза.

Шаг 2. Очистка (рефакторинг) базы данных

Листинг 35. Удаление неиспользуемого индекса

```
DROP INDEX IF EXISTS idx_customer_order_employee_unused;
```

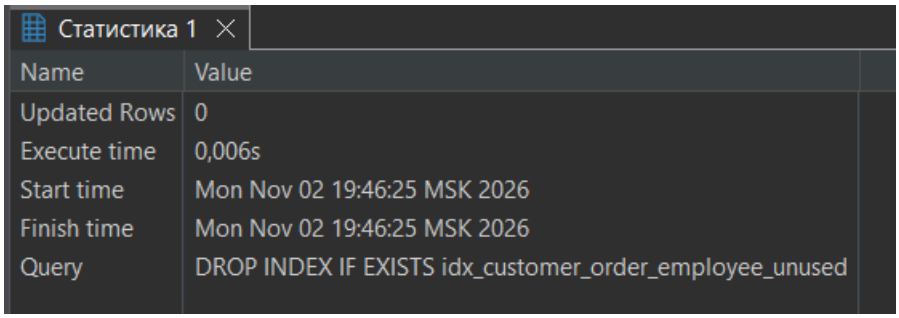


Рисунок 27. Удаление неиспользуемого индекса

Вывод. Удаление idx_customer_order_employee_unused освободило 160 kB и избавило таблицу customer_order от обновления бесполезного индекса при каждой операции записи. Каждый существующий индекс замедляет INSERT, UPDATE и DELETE, поэтому аудит индексов нужно проводить регулярно.

ВЫВОДЫ

В ходе работы на таблицах объёмом более 20 000 строк с помощью EXPLAIN (ANALYZE, BUFFERS) проанализированы запросы до и после оптимизации, рассмотрены B-Tree, Hash, GiST, BRIN и GIN индексы, покрывающий индекс и операции обслуживания базы данных. Основные результаты:

— **Простой B-Tree (п. 1.1):** число блоков снизилось с 303 до 3, Execution Time — с 2,757 до 0,114 мс.

— **Индекс по выражению (п. 1.3):** при поиске с LOWER() обычный индекс не работает; индекс по выражению сократил число блоков с 303 до 3, а время выполнения — с 11,6 мс до долей миллисекунды.

— **Составной (п. 1.2) и частичный (п. 1.4) индексы** дали скромный выигрыш по блокам (194 → 175) или не дали его (194 → 254), потому что нужные строки разбросаны по таблице и составляют заметную долю (≈33 % у частичного индекса). Эффективность индекса зависит от селективности условия и физического порядка данных.

— **Hash и GiST (п. 2.1):** Hash — для равенства (521 → 3 блока), GiST — для поиска по подсети (521 → 81 блок).

— **BRIN (п. 2.2):** индекс занимает 24 kB против 456 kB у B-Tree, но работает только на упорядоченных данных: после CLUSTER число блоков снизилось с 194 до 26.

— **Покрывающий индекс (п. 2.3)** исключает обращение к таблице ценой большего размера индекса; **GIN (п. 2.4)** ускорил поиск по

JSONB в ≈2,5 раза (12,118 → 4,862 мс), но занимает около 70 % размера таблицы.

Обслуживание базы данных (задание №3) необходимо выполнять регулярно: ANALYZE (или autovacuum) — чтобы оптимизатор опирался на актуальную статистику; VACUUM — чтобы очищать мёртвые строки после UPDATE и DELETE и поддерживать карту видимости для Index-Only Scan; CLUSTER — по мере деградации физического порядка, только в окна обслуживания из-за эксклюзивной блокировки; аудит pg_stat_user_indexes — чтобы удалять неиспользуемые индексы. В Production-среде новые индексы следует создавать командой CREATE INDEX CONCURRENTLY, чтобы не блокировать запись в таблицу.